

### Problem 2.1.5 — Reading and printing a spreadsheet document

Write a program that runs on NodeJS and reads a given file using the fs module:<https://nodejs.org/en/learn/manipulating-files/reading-files-with-nodejs>[Links to an external site.](#). The file should be formatted in CSV (see [https://en.wikipedia.org/wiki/Comma-separated\\_values](https://en.wikipedia.org/wiki/Comma-separated_values)[Links to an external site.](#) for information about CSV). Program a simple display that just prints the entire spreadsheet table as plain text on the NodeJS console. Maintain the table structure while printing it as plain text. Because the NodeJS console offers limited styling options, you should think about ways to create a good visualization for tables. One option is to insert a vertical line “|” between the cells, but these are hard to align for long numbers or text values. Another option is to print a tab “\t” character between the cells, which will work better, but still not for long numbers or text values. Yet another option is to count the digits or letters for all values in a column and then insert blank spaces around the shorter values.

You do not need to create an internal representation of the spreadsheet—that will be done in Problem 2.1.6. Just read a CSV file and print it out. The file name should be specified on the command line when the program is run. Then program a prompt to allow the user to specify the file path and display the file from the same program. The solution should include extensive Jasmine test cases, particularly for edge cases (described in Section 1.1.3). Example tests include, but are not limited to, checking for erroneous file format; “unusual” spreadsheets, such as jagged tables with uneven lengths of rows; a large spreadsheet that does not fit the screen; many blank spaces or tabs between the cells, empty cells, rows overflowing to the next line, trailing comma characters, non-numerical cell values, or spreadsheet formulas (for now show them as plain text—they will be evaluated in Problem 2.1.6); etc.

**Attach the screenshot** of your result to prove the format of the result and the readability along with Homework #1.

(You can also find HW#1 problem in PDF document: **JavaScript lecture notes \_\_2024-03-19.pdf** page 52, available on **Canvas**, under the tab "Files")

Homework submission formats: only **PDF** or **plain text** are acceptable for single documents.

**Use a single ZIP file (.zip)** if the code and tests are submitted in separate files.

**Program code quality is critical.** It is critical that the code gives a correct solution, but it is also critical that it is well designed for usability and readability. A good narrative explanation of codes and results are preferred.

**No copy of codes.** It is acceptable to collaborate on solving a problem, but it is **not acceptable** to copy each other's

solutions. **Homework solutions raising suspicion of copying will be assigned 40 % of the maximum grad or even zero grade.**